

TITANOMNIGROUP – ULTRA ENTERPRISE BLUEPRINT

Full-scale SaaS system architecture for Cursor development.

Includes modules, flows, API design, scaling, and production strategy.

1. SYSTEM ARCHITECTURE (DETAILED)

Backend: Node.js + TypeScript (NestJS recommended)

Frontend: Next.js + TailwindCSS

Database: PostgreSQL (primary), Redis (cache/queue)

Pattern: Clean Architecture (Controller → Service → Repository)

CoreEngine handles module lifecycle, dependency injection, and phase activation.

2. MODULE BREAKDOWN

Users Module: auth, JWT, roles

CRM Module: lead ingestion, tagging, filtering, pipeline

Contracts Module: document generation, lifecycle states

Billing Module: invoices, Stripe integration, subscriptions

Titanix Module: workflow automation, email sending, scheduling

Analytics Module: revenue metrics, conversion tracking

Notification Module: email + Telegram alerts

AI Assistant Module: chat logic, conversation storage, qualification

3. DATA FLOW (STEP BY STEP)

1. Lead enters via API or manual input
2. Stored in CRM with status NEW
3. Titanix processes lead → sends email
4. If response → contract created
5. Contract signed → invoice generated
6. Payment completed → revenue recorded
7. Analytics updates metrics
8. System optimizes future actions

4. API DESIGN

Auth:

POST /auth/register

POST /auth/login

CRM:

POST /leads

GET /leads

PATCH /leads/:id

Contracts:

POST /contracts

GET /contracts

Billing:

POST /invoices

GET /invoices

Dashboard:

GET /dashboard/overview

AI:

POST /ai/chat

POST /ai/session

5. DTO STRUCTURE

LeadDTO: name, email, status

ContractDTO: user_id, value, status

InvoiceDTO: contract_id, amount, status

6. DATABASE RELATIONS

User 1:N Contracts

Contract 1:N Invoices

Lead linked to user optionally

Indexes on email and status fields

7. QUEUE & BACKGROUND JOBS

Use Redis + BullMQ

Jobs:

- email sending
- analytics processing
- AI tasks

Retry logic and failure handling required

8. ERROR HANDLING

Central error middleware

Logging with levels (info, warn, error)

Retry for external API failures

9. SECURITY

JWT auth

Rate limiting

Input validation

Encryption for sensitive data

Audit logs

10. DEPLOYMENT

Docker containers:

- backend
- postgres
- redis

Use environment configs

CI/CD via GitHub Actions

11. SCALING STRATEGY

Horizontal scaling with multiple instances

Load balancer (NGINX)

Microservices transition after v4

Separate AI services if needed

12. FRONTEND STRUCTURE

Next.js pages:

/dashboard

/client

/login

Components: Sidebar, Cards, Tables

API integration via fetch/axios

13. AI ASSISTANT LOGIC

Receives user input

Processes via OpenAI

Responds with structured output

Stores conversation

Triggers CRM updates

14. PHASE DEVELOPMENT PLAN

v1: Core + CRM + Billing

v2: Automation + Email

v3: AI Assistant

v4: Analytics + Optimization

v5: Scaling + Microservices

v6+: Full ecosystem expansion

15. FINAL NOTES

System must remain modular

Focus on revenue modules first

Test before scaling

Avoid overengineering early stages